Interview Questions

Compiled from 60+ Real Interviews

Ankita Soni

Technical Lead at Persistent Systems

Ex-Goldman Sachs | Wayfair | Talkdesk

60+	100+	200+	15+
Interviews Given	Questions Documented	LinkedIn Connections	Categories Covered

Every question in this document was asked in a real interview.

Follow-up questions marked with ➔ are what interviewers ask AFTER your initial answer.

Table of Contents

1. Java Core & Collections

■ Asked in 90%+ of Java interviews. HashMap internals is the #1 most-asked question.

- How does HashMap work internally?
  - ➔ *Follow-up: How does HashMap calculate the bucket index?*
  - ➔ *Follow-up: What happens during a hash collision?*
  - ➔ *Follow-up: What changed in Java 8? (Treeification at 8 entries + capacity >= 64)*
  - ➔ *Follow-up: Why is the treeification threshold 8? (Poisson distribution)*
- Explain the hashCode() and equals() contract.
  - ➔ *Follow-up: What happens if you override equals() but not hashCode()?*
  - ➔ *Follow-up: Objects 'disappear' from HashMap — explain why.*
- HashMap vs ConcurrentHashMap — when to use each?
  - ➔ *Follow-up: How does ConcurrentHashMap work internally?*
  - ➔ *Follow-up: Java 7 segment locking vs Java 8 CAS approach — differences?*
  - ➔ *Follow-up: Does ConcurrentHashMap ever lock the entire map? (No)*
- What are Fail-Fast vs Fail-Safe iterators?
  - ➔ *Follow-up: What is a weakly consistent iterator?*
- Difference between Class, Object, and Struct in Java?

2. Java 8+ & Concurrency

■ CompletableFuture and Virtual Threads come up in every senior Java round.

- What are Functional Interfaces? Give examples.
- Explain Lambda Expressions with a real use case.
- Stream API — lazy evaluation, intermediate vs terminal operations.
  - ➔ *Follow-up: When do parallel streams hurt performance?*
- CompletableFuture — how would you merge responses from 3 APIs in parallel?
  - ➔ *Follow-up: supplyAsync vs runAsync — when to use each?*
  - ➔ *Follow-up: thenCombine() vs allOf() — differences?*
  - ➔ *Follow-up: What thread pool does supplyAsync use by default? (ForkJoinPool.commonPool)*
  - ➔ *Follow-up: Why is using the common pool dangerous? How to fix? (Custom ExecutorService)*
  - ➔ *Follow-up: What happens if one API times out? (.orTimeout, .completeOnTimeout)*
- Future vs CompletableFuture?
  - ➔ *Follow-up: Why is Future.get() blocking a problem?*
- Explain ExecutorService and Thread Pools.
- Core multithreading concepts (synchronization, volatile, atomic operations).
- What are Virtual Threads in Java 21? How do they differ from platform threads?
  - ➔ *Follow-up: When would virtual threads NOT be the right choice?*
  - ➔ *Follow-up: What is Project Loom?*

3. Spring Boot & Hibernate

■ **@Transactional proxy bypass (same-class call) catches 90% of candidates.**

- @Component vs @Service vs @Repository — does it actually matter?
- What happens when a Spring Boot application starts? Walk through the flow.
  - ➔ *Follow-up: How does auto-configuration decide which beans to create? (spring.factories)*
- Explain Dependency Injection in Spring.
- Bean Lifecycle — creation to destruction.
- @Autowired internals — how does Spring resolve dependencies?
- BeanNotFoundException — common causes and how to fix?
- @Transactional — what actually happens under the hood?
  - ➔ *Follow-up: What happens when you call a @Transactional method from WITHIN the same class?*

■ **It bypasses the proxy. Transaction doesn't apply. Top 3 most-asked Spring question.**

- Lazy Loading vs Eager Loading in Hibernate — trade-offs?
  - ➔ *Follow-up: How do you handle lazy loading in a REST API response? (LazyInitializationException)*
- Explain the N+1 Query Problem — detect, explain, fix.
  - ➔ *Follow-up: How to detect: EXPLAIN ANALYZE, Hibernate SQL logging.*
  - ➔ *Follow-up: Fixes: JOIN FETCH, @EntityGraph, @BatchSize.*
- Hibernate Caching — first level vs second level cache.
- Build a Spring Boot CRUD app with an in-memory rate limiter (10 req/min, return 429).

■ **Tests 5 skills at once: REST APIs, HTTP codes, rate limiting algo, thread safety, separation of concerns.**

4. Database & SQL

- What is a database index? Trade-offs on write-heavy tables?
  - ➔ *Follow-up: What is cardinality and why does it matter for index selection?*
- Clustered vs Non-Clustered Index — when to use each?
  - ➔ *Follow-up: Table with 50M rows, queries slow. Walk through your investigation.*
- Normalization vs Denormalization — when to use which?
- Partitioning vs Sharding?
  - ➔ *Follow-up: How do you choose a shard key? (High cardinality, even distribution)*
- ACID Properties — explain each with real examples.
- Optimistic vs Pessimistic Locking?
  - ➔ *Follow-up: E-commerce inventory updates — which one and why?*
- How to debug a slow database query?
  - ➔ *Follow-up: Reading EXPLAIN plans — what to look for?*
  - ➔ *Follow-up: Full table scan causes and how to eliminate them?*
- SQL query optimization techniques.
- Why use 2 separate DBs for read and write? (CQRS)
  - ➔ *Follow-up: How to manage consistency between read and write stores?*

5. Microservices & Distributed Systems

■ **'Why doesn’t @Transactional work across services?' is the #1 microservices question.**

- Monolith vs Microservices — trade-offs?
- Monolith to Microservices migration strategies?
- Explain the Strangler Fig Pattern.
- API Gateway — what is it and why is it needed?
- Service Discovery — how do services find each other?
- Database-per-Service pattern — why and how?
- Why doesn't @Transactional work across microservices?
  - ➔ *Follow-up: What are Distributed Transactions? Why do they fail at scale?*
  - ➔ *Follow-up: Two-Phase Commit (2PC) — why is it problematic?*
- Explain the Saga Pattern.
  - ➔ *Follow-up: Choreography vs Orchestration Saga — when to use which?*
  - ➔ *Follow-up: What about compensation transactions?*
  - ➔ *Follow-up: What tools support orchestration? (Temporal, AWS Step Functions, Camunda)*
- Event-Driven Architecture — explain with a real example.
  - ➔ *Follow-up: How do you ensure data consistency with events? (Transactional Outbox Pattern)*

6. Resiliency & Reliability Patterns

- Circuit Breaker Pattern — explain the three states.
  - ➔ *Follow-up: How is a circuit breaker different from a retry?*
  - ➔ *Follow-up: What tool do you use? (Resilience4j, not Hystrix — it's deprecated)*
- Retry Pattern — when to retry and when NOT to?
- Exponential Backoff — why not just retry immediately?
  - ➔ *Follow-up: What is Exponential Backoff with Jitter? Why add jitter?*
- Bulkhead Pattern — what problem does it solve?
- Timeout Pattern — how to set appropriate timeouts?
- Dead Letter Queue (DLQ) — when and why?
  - ➔ *Follow-up: DLQ vs just logging errors — why DLQ is better?*

7. Messaging Systems

- Kafka vs RabbitMQ — when to use each?

■ **Kafka = event STREAMING (replay). RabbitMQ = message BROKER (task delivery).**

- When to use Kafka? When to use RabbitMQ?
- Explain Consumer Groups in Kafka.
- Partitions — how do they enable horizontal scaling?
- Offsets — what happens if a consumer crashes before committing?
- Delivery Guarantees — at-most-once, at-least-once, exactly-once?
  - ➔ *Follow-up: Which one do you use in production and why? (Usually at-least-once + idempotent consumers)*
- What is the Poison Pill problem in Kafka?
  - ➔ *Follow-up: How do you distinguish transient vs permanent failures?*
- Transactional Outbox Pattern — why and how?
  - ➔ *Follow-up: Polling Publisher vs CDC (Debezium) — trade-offs?*
  - ➔ *Follow-up: What about duplicate events? (Idempotent consumers)*
  - ➔ *Follow-up: What about event ordering? (Same partition key)*

8. AWS & Cloud

- What is Serverless?
- ECS vs Lambda — when to choose which?
  - ➔ *Follow-up: Is ECS serverless? (Fargate = yes, EC2 = no)*
  - ➔ *Follow-up: Lambda cold start — when is it a problem?*
- ECS vs Fargate?
- Lambda internals — how does it work under the hood?
- SQS vs SNS — when to use each?

■ **SQS = one consumer queue. SNS = fan-out to many. Common: SNS → multiple SQS queues.**

  - ➔ *Follow-up: What is a Dead Letter Queue in SQS?*
- API Gateway — how does it handle millions of requests?
- EventBridge — when to use over SNS?
- CloudWatch — metrics, logs, alarms.
- Auto Scaling — how does it work?
- Load Balancers — ALB vs NLB?
- Design an email notification system using AWS services.
  - ➔ *Follow-up: How to handle 1M emails/hour?*
  - ➔ *Follow-up: Failure handling — DLQ strategy?*
- Scheduled email notifications from DB records — design?
- Handling millions of requests through API Gateway?

9. NoSQL Databases

- MongoDB vs DynamoDB — when to use each?
- When to use MongoDB? When to use DynamoDB?
- Partition Keys in DynamoDB — how to choose?
- Replica Sets in MongoDB — primary/secondary roles, election process.
- Sharding in MongoDB — when to shard, how to pick shard keys?
- CAP Theorem — explain with real-world examples.

■ **DynamoDB = AP. PostgreSQL = CP. 'CA' doesn't exist in distributed systems.**

- Eventual Consistency — what does it actually mean?
  - ➔ *Follow-up: When is eventual consistency acceptable? When is it not?*

10. React & Frontend

- React Lifecycle — mounting, updating, unmounting phases.
- Functional vs Class Components — why functional won?
- useState — how does it work under the hood?
- useEffect — dependency array, cleanup function, common pitfalls.
- useMemo vs useCallback — explain the difference.
  - ➔ *Follow-up: When does useMemo actually HURT performance?*
- React.memo — when to use and when to avoid?
- Context API vs Redux — when to choose which?
  - ➔ *Follow-up: What about Zustand or Jotai? (Modern alternatives)*
- How does React reconciliation work? What is the Virtual DOM actually doing?
  - ➔ *Follow-up: When does React bail out of re-rendering?*

React Routing

- Client-side vs Server-side routing — trade-offs?
- BrowserRouter vs HashRouter?
- Dynamic Routes, Nested Routes, Protected Routes, Lazy Loaded Routes.

Frontend Performance

- How would you optimize a React app rendering 10,000 list items?
  - ➔ *Follow-up: Virtualization (react-window), code splitting, React.lazy + Suspense.*
- Debouncing vs Throttling — implement from scratch.
  - ➔ *Follow-up: When does debounce hurt UX? (Submit buttons)*
- Image optimization, CDN usage, state management optimization.
- SSR vs CSR vs SSG — when to use each?
  - ➔ *Follow-up: What is hydration?*
  - ➔ *Follow-up: How does Next.js let you mix all three?*

11. JavaScript Fundamentals

- How does asynchronous processing work in JavaScript?
- Explain the Event Loop — Call Stack, Web APIs, Callback Queue, Microtask Queue.
  - ➔ *Follow-up: Output: console.log(1); setTimeout(()=>console.log(2),0); Promise.resolve().then(()=>console.log(3)); → 1, 3, 2*
- Promises vs Async/Await — differences and when to use which?
- Explain Closures with a practical example.
  - ➔ *Follow-up: Classic closure problem with var vs let in for loops.*
- Event Bubbling vs Event Capturing?
- Event Delegation — why is it useful?
- stopPropagation() — when to use?
- Type Coercion in JavaScript — explain with gotchas.
- == vs === — when to use which?
- delete vs splice — differences?
- 'this' keyword in different contexts (global, method, arrow function, class).
- JavaScript output-based questions (common in interviews).

12. TypeScript

- What are the differences between type and interface?
  - ➔ *Follow-up: When to use type vs interface? (type for unions/intersections, interface for objects/contracts)*
- Explain Generics with a real example.
  - ➔ *Follow-up: Utility types: Partial, Pick, Omit, Record — when to use each?*
- What are Decorators in TypeScript?
- Implement a Promise polyfill from scratch.
  - ➔ *Follow-up: Handle async callbacks, make it chainable, internal state, callback queue, static methods.*

13. Web Fundamentals & Browser

- Explain browser components: JS Engine, Rendering Engine, Call Stack, Web APIs, Event Loop.
- How does the browser render a page? (Critical Rendering Path)
- What happens when you type a URL and hit Enter?

■ **DNS → TCP → TLS → HTTP → Server → Rendering. Classic question.**

- sessionStorage vs localStorage vs cookies — differences?
- What is CORS and how does it work?
  - ➔ *Follow-up: Preflight request (OPTIONS) — when does it happen?*
- HTTP/2 vs HTTP/1.1 — key differences?
  - ➔ *Follow-up: Multiplexing, header compression, server push.*

14. System Design / HLD

■ **Every SD round tests: clarify requirements → estimate scale → data model → APIs → HLD → deep dive.**

Common System Design Problems

- Design a URL Shortener (TinyURL)
  - ➔ *Follow-up: How to handle 10B redirects/month? 301 vs 302?*
- Design a Notification System
  - ➔ *Follow-up: Email vs SMS vs Push — different latency requirements?*
- Design a Rate Limiter
  - ➔ *Follow-up: Token Bucket vs Sliding Window? API Gateway vs app layer?*
- Design a Payment System
- Design a Parking Lot System
- Design a Bike Rental System
- Design a Logging Framework
- Design an Employee Management System
- Design a Hotel Availability & Pricing System
- Design a Real-time Ad Bidding System

System Design Concepts

- Caching Strategies — cache-aside, write-through, write-behind?
- Redis — when and how to use? (Never just say 'for caching')
- CDN Architecture — how does it work?
- Read Replicas — when and why?
- Consistent Hashing — explain with examples.
- Horizontal vs Vertical Scaling — trade-offs?

The Debugging Question (asked in 80% of rounds)

- Your API returns 5-second response times under load. Walk through debugging.

■ **Start with observability, not 'add a cache.' APM → EXPLAIN ANALYZE → thread pools → circuit breakers.**

15. DevOps & CI/CD

- How would you add authentication to a REST API?
  - ➔ *Follow-up: JWT vs sessions in microservices? Why stateless?*
- How did you use Infrastructure as Code?
  - ➔ *Follow-up: Biggest risk with IaC? (Drift)*
- Explain your CI/CD pipeline end to end.
  - ➔ *Follow-up: What if production deploy breaks? (Blue/green rollback)*
- Blue/Green vs Canary deployment — when to use which?
- Which tools do you use for deployment?
- How are Kubernetes deployments done?
  - ➔ *Follow-up: Rolling update strategy? Readiness probes?*

16. DSA — Scenario-Based Questions

■ **They didn't say 'use a heap.' They described a PROBLEM and expected me to pick the data structure.**

- Gaming platform: 1M users. Check if username is taken instantly.
  - ➔ *Follow-up: Answer: HashSet. O(1). Memory concern? Bloom Filter.*
- Build cache: max 10K items, evict LRU, all operations O(1).
  - ➔ *Follow-up: Answer: HashMap + Doubly Linked List.*
- Stock app: thousands of price updates/sec, show top 10 most expensive.
  - ➔ *Follow-up: Answer: Min-Heap of size 10. Why min, not max? Remove SMALLEST of top 10.*
- Browser history with back and forward navigation.
  - ➔ *Follow-up: Answer: Two Stacks. New navigation clears forward stack.*
- Implement a Queue using Stacks.
  - ➔ *Follow-up: Answer: Two stacks. Amortized O(1).*

17. Behavioral / Experience-Based

- Tell me about a production issue you debugged.
- Give an example of performance optimization you did.
- How did you reduce API latency in a real project?
- Describe a scalability challenge you faced.
- Tell me about a system migration you led.
- Describe your experience with microservices architecture.
- Explain a project using AWS + Kafka + Spring Boot + React.
- Tell me about a time you owned a product end-to-end (PRD to production).
- Describe a time you disagreed with a technical decision. What did you do?

Connect with me on LinkedIn for more interview prep content.

linkedin.com/in/ankita-soni